

AI Requirements and Taxonomy Guide

What a Taxonomy Is

A taxonomy is an agreed set of categories you sort things into, and a consistent way you label them. That is all.

In the context of an AI-assisted project, a requirements taxonomy means:

- A short list of requirement types (functional, security, data, and so on).
- A consistent naming pattern for each requirement (FR-CRM-001, BR-OPS-002).
- A simple priority scheme (Must / Should / Could / Won't).
- A status system to track where each requirement is in its lifecycle.

You are deciding: when I write a requirement, which category does it belong to, and how do I label it so I can find it later?

Why Classification Matters

Without a taxonomy, requirements accumulate as an undifferentiated list of bullet points. This creates several problems:

- Different types of requirements get mixed together, making it hard to see gaps.
- AI tools have no reliable way to know which requirements govern which part of the system.
- Reviews lack focus. A security reviewer has to wade through functional requirements to find the security ones.
- Traceability breaks down. You cannot easily ask “what are all the data requirements?” or “what has changed in the security requirements since last month?”

A taxonomy does not have to be elaborate. Even a basic classification system makes a significant difference as a project grows.

[!NOTE]

This guide exists because AI-assisted projects often generate a large volume of requirements quickly. Without a classification system, that volume becomes unmanageable.

Suggested Requirement Types

Use the following types as a starting point. Adapt them to your project if needed.

Prefix	Type	What It Covers
FR	Functional	What the system or product must do: features and behaviours.
BR	Business Rule	Rules that must always be enforced, regardless of how they are implemented.
DR	Data	What data is stored, required fields, quality rules, and retention.
SR	Security	Access control, encryption, authentication, and data protection.
AR	Audit/Traceability	Logging, evidence, version history, and approval records.
AIR	AI/Guardrails	How and where AI is used, what it must and must not do, and human approval points.
CR	Configuration	What can be configured, by whom, and how it is controlled.
IR	Integration	Connections to other systems, data flows, and failure handling.
RR	Reporting/Output	Reports, dashboards, alerts, and exported documents.
NFR	Non-Functional	Performance, availability, usability, scalability, and accessibility.

You do not need all of these in every project. Use the types that are relevant and skip the rest.

Modular Requirement IDs

For small projects, a simple ID like FR-001 is sufficient. As projects grow and span multiple modules or business capabilities, modular IDs become valuable.

A modular ID has three parts:

```
[ TYPE ] – [ MODULE ] – [ SEQUENCE ]
```

Examples:

- FR-CRM-001: The first functional requirement in the CRM module.
- BR-OPS-002: The second business rule in the Operations module.
- SR-SEC-003: The third security requirement in the Security module.
- AIR-AI-004: The fourth AI guardrail requirement in the AI module.

The module code is a short abbreviation of the business capability or project area. Define your module codes at the start of a project and use them consistently.

[!NOTE]

Sequences are zero-padded to three digits (001, 002, 003) and increment within each type-module combination. They do not reset when you start a new document.

Priority Values

Use MoSCoW notation for all requirement priorities.

Priority	Meaning
Must	Non-negotiable. The project fails without this.
Should	High value. Include unless a strong reason exists not to.
Could	Desirable. Include if time and budget allow.
Won't	Explicitly out of scope for this release.

Recording Won't requirements is as important as recording Must requirements. Explicit exclusions prevent scope creep and limit implied expectations.

Requirement Lifecycle Status

Track each requirement through its lifecycle.

Status	Meaning
Proposed	Identified but not yet formally drafted.
Draft	Written but not yet reviewed.
Under Review	In active review.
Approved	Accepted and baselined.
Deferred	Out of current scope, retained for a future phase.
Rejected	Explicitly excluded, with documented rationale.
Implemented	Delivered and verified.
Retired	No longer applicable; superseded or removed.

Source Classification

Recording where a requirement came from supports compliance mapping and traceability. This is particularly useful for regulated industries or client engagements where you may need to demonstrate that certain requirements trace back to a specific obligation.

Source	Meaning
Legislative	Derived from a specific law or regulation.
Regulatory	Derived from a regulatory standard or guidance.
Policy	Derived from internal or client policy.
Operational	Derived from operational need or workflow analysis.
Client-driven	Requested directly by a client.
Security	Identified through security analysis or risk assessment.
AI Governance	Arising from AI use or model risk considerations.
Audit Finding	Arising from an audit, review, or assurance activity.
User Feedback	Arising from user research, testing, or direct feedback.

Related Decisions, Modules, and Issues

Each requirement can carry explicit links to related artefacts. This creates a traceable web of connections across the project.

Use this structure within each requirement record:

```
Related Requirements:
- SR-SEC-002
- AR-AUD-004

Related Decisions:
- DEC-014

Related Modules:
- crm-module
- reporting-module

Related Issues:
- ISS-007
```

Cross-references are most valuable where a requirement depends on, conflicts with, or directly enables another requirement or decision. Mandatory cross-referencing in those cases prevents hidden dependencies from creating problems later.

Assumptions Register

Keep a short list of assumptions that underpin your requirements. Assumptions are not decisions. They are things you have taken to be true without explicit confirmation.

When an assumption is eventually validated or invalidated, update the register and review any requirements that depended on it.

How Taxonomy Helps AI-Assisted Projects

When AI tools work with well-classified requirements, the quality of their output improves significantly.

An AI tool asked to review security requirements produces a better review when it can see which requirements are actually security requirements, rather than searching through an undifferentiated list. An AI tool generating test cases from functional requirements is more accurate when the functional requirements are clearly separated from business rules.

Classification also makes it easier to ask focused questions: “Are there any gaps in the data requirements for the reporting module?” is answerable. “Have we covered everything?” is not.

Version History

Version	Date	Author	Summary
0.1	2026-05-28		Initial draft
1.0	2026-06-13	Jason Masters/Claude	Initial Version